CMU 18-643: Reconfigurable Logic: Technology, Architecture and Applications

Handout #7/Lab 3: Hardware Accelerated Computation (250 points)

Issued: 10/5/2026

Submit by Monday, 11/2/2026

This lab must be completed in a team of 3. In this lab, you will implement a complete CNN layer on the Ultra96 Zynq SoC using a Vitis OpenCL-based flow. Be prepared and start thinking early. Please post questions and answers on 18643's Piazza page to help each other out with tools-related issues. Keep your good ideas to yourself.

On paper, you have 4 calendar weeks for this lab. Please note that the midterm, fall break, project proposal, and paper presentation are all due during the same period. Manage your time carefully.

Part 1: Getting started

The starter files in 18643_cnn_src.zip use the same C++ source code base you worked on in Lab 2, except **__VITIS_CL__** is defined in **util643.h**. In order to increase arithmetic intensity over costly DRAM accesses, the CNN kernel provided to you in krnl_cnn.cpp is a blocked version of the canonical CNN loop nests, corresponding to Figure 9 in Zhang'2015. In Lab 2, you focused on the blocked inner loop kernel that operates on BlockRAM input and output buffers. In Lab 3, the FPGA kernel expands to address the full layer with input and output buffers in DRAM. As such, the outer level of the kernel includes the code to orchestrate the buffer preparation and the execution of the block kernel you worked on in Lab 2. (This orchestration code was a part of the testbench in Lab 2)

To get started

1. Set up Vitis environment:

   `source /afs/ece.cmu.edu/class/ece643/software/scripts/setup_vitis_fall26.sh`

2. Create a workspace: `cd /scratch/643_vitis_<AndrewID>; mkdir lab3`

3. Unpack source files: `cd /scratch/643_vitis_<AndrewID>/lab3; unzip <18643_cnn_src.zip>`

Using **/scratch/643_vitis_<AndrewID>/lab3** as your workspace, create a Vitis project for **cnn** following the same recipe for **vadd** in **Lab 0**. Note the following deviations:

1) **Be sure to add the Application component before adding the HLS component.**
2) Name the Application component **cnn_host**.
3) Import **cnn.cpp**, **util643.h**, **kernel643.h**, and **instance643.h** in **/scratch/643_vitis_<AndrewID>/lab3/18643_cnn_src/** as Application Sources.
4) Name the HLS component **krnl_cnn**.
5) Add **krnl_cnn.cpp** in **/scratch/643_vitis_<AndrewID>/lab3/18643_cnn_src/** as Design Files.
6) Add **util643.h**, **kernel643.h**, and **instance643.h** in **/scratch/643_vitis_<AndrewID>/lab3/cnn_host/** also as Design Files. (Note that here you are adding from the application component directory **cnn_host/**, not the unpack directory **18643_cnn_src/**. This way, the Application and the HLS components point to the same .h files.)

7) Select **krnl_cnn()** as the Top Function for synthesis.
8) Set target clock to **6.6ns** and uncertainty to **27%**. You can change clock and uncertainty during performance tuning. (The initial 6.6ns is chosen to match the default 150MHz clock. The HLS compiler will try to fit within *period**(1-*uncertainty*) to improve the likelihood of place-and-route timing closure at *period*.)
9) Create a system component named **cnn**; and name the binary container **cnn** also.

Build and run the **cnn** project on your Ultra96v2. Note the execution time and performance reported as your baseline.

A note on adjusting acceleration kernel clock frequency.

Setting the clock target of the HLS component tells the HLS compiler how hard to optimize. It does not control the clock finally used to drive your kernel. Vitis by default assigns Ultra96v2's 150MHz PL clock 0 to kernels. If your HLS kernel design isn't fast enough, it will fail place-and-route timing closure. In this case, you can either specify a tighter HLS timing target or select a slower fabric clock. You can also try for a higher clock frequency if you have timing slack.

The following clocks are selectable as alternative:

- PL0: 150MHz
- PL1: 300MHz
- PL2: 75MHz
- PL3: 100MHz
- PL4: 200MHz
- PL5: 400MHz
- PL6: 600MHz

To select them instead of the default:

- In the Component Explorer, click on the **cnn** system component and then on **vitis-sys.json**.
- Under the Hardware Linker Settings, click on the caret next to the **cnn** container.
- Click on **hw_link/cnn-link.cfg**.
- Click on the "</>" symbol at the top to switch into text editing mode.
- Add the following 2 lines with **id** set to your clock choice (e.g., 2 for 75MHz):

```
[clock]
id=2:krnl_cnn_1
```

Part 2: Let's see what you can do.

The goal of this lab is to improve the throughput of computing a CNN layer (as configured in instance643.h) on a batch of 10 inputs. The timing measured and reported by **main.cpp** includes host-accelerator data transfer overhead incurred at the start and end of the batch. ****You have full flexibility to modify krnl_cnn.cpp and kernel643.h. You may not add or modify other files in the final submission.****

To do:

Step 1: Understand krnl_cnn(). This lab is primarily about managing the usage of in-fabric BRAM capacity (~1 Mbyte) and the off-chip DRAM bandwidth (~4 GB/sec best case). It is imperative that you understand thoroughly **krnl_cnn()** (based on Figure 9 of Zhang'2015). Therefore, read Zhang'2015 again

carefully. Afterwards, return to your finished Lab 2 project. Build the Lab 2 source code as a regular C++ program in your favorite C++ environment. In a debugger, step through an execution to review how the code in **krnl_cnn()** is organized and traversed.

Step 2: Decide on an execution strategy. Although HLS can shield you from the low-level hardware datapath details, a performance tuning discipline based on understanding and exerting control over how/when/where compute, data buffering, and data movement take place is nevertheless necessary to achieve performance. (This is true whether you are using Verilog or HLS to develop on an FPGA; this is also true whether you are developing for performance on an FPGA, GPU, or any spatially concurrent platform.)

Based on what you have seen in lectures and earlier labs, visualize at a conceptual level what should happen—that is, how/when/where compute, data buffering, and data movement take place. All basic strategies should result in the same total number of arithmetic operations ($2\times M\times N\times R\times C\times K\times K$) in a layer. Performance difference between two strategies arises mainly from their differences in (1) the number of times a data element (weights or feature map) is copied between DRAM and fabric; (2) the DRAM access pattern when transferring data elements between DRAM and fabric; (3) how data on-chip are buffered and accessed; and (4) the number of concurrent arithmetic operations per cycle when not waiting for data transfers.

Step 3: Implement your strategy.
Follow the lowest-hanging-fruit-first principle when optimizing performance. It is useful to review the concepts of latency, throughput, overhead, amortization, latency hiding, as well as the memory lectures. At each refinement iteration, first determine the performance-limiting bottleneck to be addressed next.

You want to reconsider the design and optimization decisions of **cnn_blocked_kernel()** made in Lab 2. You do not need to preserve the **cnn_blocked_kernel()** function boundary. The variables are named such that you can replace the call to **cnn_blocked_kernel()** in **krnl_cnn()** by **cnn_blocked_kernel()**'s body block directly.

An important degree of freedom is the multi-dimensional array layout of the weights and feature map in DRAM. You can alter the data layout by adjusting the macros **ARRAYi**, **ARRAYo**, and **ARRAYw** in **kernel643.h**. It is important the code in **krnl_cnn()** access the in-DRAM weights and feature maps using these macros since the host code uses these macros to prepare the data buffers.

You want to develop your code in the most expedient way possible. This means first as a regular C++ project, then in the HLS environment, and only finally on real hardware.

- At any time, you can copy all of the .cpp and .h files from **/scratch/643_vitis_<AndrewID>/lab3/cnn_host/** and **/scratch/643_vitis_<AndrewID>/lab3/18643_cnn_src/krnl_cnn.cpp** to a temporary directory to build and debug as an ordinary C program by commenting out **__VITIS_CL__** in **util643.h**.
- Once you are sure your C++ code is functionally correct, you should next attempt SW-emulation to double check everything is functionally correct in Vitis. To return to working in Vitis,

reintroduce the .cpp and .h files back to the workspace and remember to restore **__VITIS_CL__** in **util643.h**.

- After successful SW-emulation of functional correctness, you can begin kernel performance tuning of the HLS component. You should expect to iterate this and the previous step several times.
- You should attempt to build and run on the Ultra96v2 only after you are satisfied with everything you can do without place-and-route. Expect to be surprised in the first few real hardware runs.

Submit: Create a submission directory **<team_name>_lab3/** to turn in the artifacts requested. <team_name> should be a concatenation of the team members' AndrewIDs in alphabetical order connected by '_'. One member of a team should submit a single file called **<team_name>_lab3.zip** (that is the zip of your submission directory) through Canvas.

Include in the submission directory a zip of your complete workspace directory after clearing the build artifacts (by clicking the "broom" symbol to the right of **Build All**).

Include in the submission directory a drawn block diagram of the synthesized kernel datapath.

Complete the Lab 3 Canvas questionnaire discussing the following.

1. Explain your final execution strategy (how it works? why did you choose to do it this way?)
2. Discuss any ideas you tried but not adopted for the final implementation.
3. Report your runtime and ops/sec (as reported by the test wrapper) for a batch of 10 inputs.
4. Find and summarize the fabric resource utilization.
5. Analyze and report the following:
    a. The average (that is, total value for the batch divided by **N**) number of times a data value is transferred between DRAM and fabric---count separately DRAM reads and DRAM writes and separately for weights, input feature maps and output feature maps. (6 numbers for each layer.)
    b. The average number of arithmetic operations performed for each data value transferred to or from DRAM. (This is arithmetic intensity.)
    c. The average DRAM bandwidth (read and write together) utilized.
6. Discuss the difference between the estimated peak performance of **cnn_blocked_kernel()** in Lab 2 vs in Lab 3. What motivated the design changes, if any, from Lab 2 to Lab 3 (e.g., re-tuned **Tm**, **Tn**, **Tr**, **Tc** and different pragmas) in the blocked kernel?
7. Discuss the difference between the achieved end-to-end performance in Lab 3 vs the estimated peak performance of **cnn_blocked_kernel()** in Lab 3. Is your overall design efficient/balanced?
8. Discuss what and why you would do differently if you had more time for another try.
9. At this moment, would you consider using Vitis for your project (yes or no)?

Grading: 60% of the grade is subjective based on the quality of the effort as represented by the files submitted. The other 40% is based on the throughput achieved.

You will receive the full 40% if you achieve greater than 8x the performance of the starter project (unmodified and run at its maximum valid frequency). You will receive 30% if you achieve greater than 4x the performance of the starter project. You will receive 20% if you achieve greater than 2x the performance

of the starter project. You will receive 10% if you achieve greater than 1.5x the performance of the starter project. Otherwise, you will receive 0%.